191.002 VU Betriebssysteme

EXERCISE 3

Last update 2025-11-05 (Version 6f3271b3)

http

HTTP is a text-based request-response application layer protocol, which is used to transfer files between a client and a server. Typically a client sends a request for a file located on the server. The server then responds and transmits the requested file.

HTTP requests start with a line which contains the request method (for instance `GET`), the location of the desired resource and the protocol version. This line is followed by an arbitrary number of header fields. Each header field is a line with a keyword and a value, separated by a colon (`:`). The request header ends with an empty line, which may be followed by a request body if the client wishes to transmit a resource to the server.

```
GET /index.html HTTP/1.1
Host: www.myhost.at
```

Example of a HTTP request: Request method `GET` is used to request the file `index.html` using HTTP version 1.1. The request header contains only one header field, the `Host` field, which specifies the hostname of the server. An empty line ends the header and since there is no body this is also the end of the request message.

The response starts with a line containing the protocol version, a status code and a textual description of the status code. As in the request, this line is followed by an arbitrary number of header fields and again the header ends with an empty line. The header is followed by the response body, which is typically the content of the requested file.

```
HTTP/1.1 200 OK
Date: Sun, 11 Nov 18 22:55:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 146
Last-Modified: Mon, 01 Oct 18 12:15:00 GMT
Connection: close

<!DOCTYPE html>
<html>
  <head>
    <title>My website</title>
  </head>
  <body>
    <h1>Hi!</h1>
    <p>This is my website.</p>
  </body>
</html>
```

Example of a HTTP response: The server responds to the above request with status code 200, which indicates that the request was successful. The header contains several fields with information about the content, such as the file format, the file length or the last time it was modified. The header ends with an empty line, which is followed by the file content.

The HTTP specification requires that each line ending in the header consists of two characters, a carriage

return followed by a linefeed character[1], instead of the linefeed only which is standard on UNIX systems. In a C-string this sequence can be encoded as `"\r\n"`. Note that this does not apply to the content in the response body, which is transmitted as it is read from the file without modifications.

Assignment A – HTTP Client

Write a client program which partially implement version 1.1 of the HTTP. The client takes an URL as input, connects to the corresponding server and requests the file specified in the URL. The transmitted content of that file is written to `stdout` or to a file.

```
SYNOPSIS
    client [-p PORT] [ -o FILE | -d DIR ] URL
EXAMPLE
    client http://www.example.com/
```

The client may be called with a number of options:

- Option `-p` can be used to specify the port on which the client shall attempt to connect to the server. If this option is not used the port defaults to 80, which is the standard port for HTTP.
- Either option `-o` or `-d` can be used to write the requested content to a file. Option `-o` is used to specify a file to which the transmitted content is written. Option `-d` is used to specify a directory in which a file of the same name (without the directory part, i.e. only the part after the last `/`) as the requested file is created and filled with the transmitted content. If the URL ends with `/` (thus requesting a directory) the filename is assumed to be `index.html`. If none of these options is given, the transmitted content is written to `stdout`.

The client creates a TCP/IP socket and connects to the hostname specified in the URL. For the purpose of this exercise we only consider URLs which start with the scheme identifier `http://`, immediatly followed by a hostname. For simplicity you may therefore assume that the hostname is the part after the initial `http://` and before any of the following characters:

```
; / ? : @ = &
```

Once the connection is established, the client sends a request for the file specified in the URL (the filepath is the part starting at and including the first `/` after the initial `http://`) using the HTTP method `GET`. The request header must start with the request line and must contain the `Host` field, which gives the hostname as specified in the URL. Also, the header field `Connection` with the value `close` should be added to the header (which tells the server that the connection should be closed after transmitting one file). For instance, the request header for the URL `http://www.example.com/` is:

```
GET / HTTP/1.1
Host: www.example.com
Connection: close
```

After transmitting its request, the client waits for a response from the server. Your client must correctly parse the status code in the first line of the response.

If the response header is invalid, the client prints the message `"Protocol error!"` to `stderr` and terminates with exit code 2. You can consider the response header to be invalid if the first line does not start with `HTTP/1.1` or if this is not followed by a number which can be parsed with `strtol(3)`.

[1] RFC 2616, Section 6: Response, `https://www.rfc-editor.org/rfc/rfc2616#section-6`

If the response status in the first line of the response header is not 200, the client prints a message with the response status (status code and the textual description after it) to `stderr` and terminates with exit code 3.

The client may ignore any header fields and directly skip ahead to the content in the response body by looking for the empty line which ends the header. The remainder of the response is the content of the requested file and is eiter written to `stdout` or to a file, according to the options listed above. The client must not rely on the response header field `Content-Length` in order to determine the amount of data transfered, but instead read from the socket until the server closes the connection.

Assignment B – HTTP Server

Write a server program which partially implement version 1.1 of the HTTP. The server waits for connections from clients and transmits the requested files.

```
SYNOPSIS
    server [-p PORT] [-i INDEX] DOC_ROOT
EXAMPLE
    server -p 1280 -i index.html ~/Documents/my_website/
```

The server may be called with following options:

- Option `-p` can be used to specify the port on which the server shall listen for incoming connections. If this option is not used the port defaults to 8080 (port 80 requires root privileges).
- Option `-i` is used to specify the index filename, i.e. the file which the server shall attempt to transmit if the request path is a directory. The default index filename is `index.html`.

The argument `DOC_ROOT` is the path of the document root directory, which contains the files that can be requested from the server.

The server listens for connections on the specified port and upon accepting a connection from a client, it reads the request message from the connection socket. Your server must be able to correctly identify the request method and the path of the requested resource.

If the request header is invalid, the server sends a response header with status code 400 (`Bad Request`) and closes the connection. You can consider the request header to be invalid if the first line does not consist of 3 values separated by whitespaces: first the request method, second the path of the requested file and third the protocol and version identifier `HTTP/1.1`.

The server must support the request method `GET`. If a different request method is sent by a client, the server sends a response header with status code 501 (`Not implemented`) and closes the connection.

If the first line of the request message is valid, the server may ignore all the rest of the request message and directly proceed to transmitting the requested file. The server prepends the document root path `DOC_ROOT` to the requested path[2]. If the path of the requested resource ends with a `/`, the default index filename (see options above) is appended to it. If the server cannot open the resulting filepath, it sends a response header with status code 404 (`Not Found`) and closes the connection.

If the file was successfully opened, the server first sends a response header with status code 200 (`OK`) and at minimum the fields `Date`, which contains the current date and time expressed in UTC (often also referred to as GMT) and encoded as specified in RFC 822[3], `Content-Length`, which gives the size of the requested file in bytes, and `Connection`, which should contain the value `close`.

[2]For the purpose of this exercise you may simply concatenate the root path and the request path, for instance by using `strncat(3)`. Your server is not required to check whether the resulting path effectively lies within the document directory.

[3]RFC 822, Section 5: Date and Time Specification, `https://tools.ietf.org/html/rfc822#section-5`

Your server may for instance transmit a response header as follows:

```
HTTP/1.1 200 OK
Date: Sun, 11 Nov 18 22:55:00 GMT
Content-Length: 146
Connection: close
```

Once the header is transmitted, the server proceeds with the transmission of the requested file, by reading the content of the file and writing it to the connection as is without any modifications. When the server is done with the transmission it closes the connection and waits for the next connection.

The response header transmitted in case of an invalid request, unsupported request method or missing file must contain the header field `Connection` with the value `close`, but all other header fields can be omitted and no content is transmitted.

The server is not required to accept more than one connection at once. Therefore, you may simply accept the next connection after the previous connection has been closed.

The server must handle the signals `SIGINT` and `SIGTERM`. If any of these signals is received, the server completes an ongoing request and then terminates with exit code 0. If there are no open connections to clients when receiving the signal, the server terminates immediately with exit code 0.

Hints (for both assignments)

- When parsing the URL in order to extract the hostname and the filepath, the functions `strchr(3)`, `strpbrk(3)` or `strsep(3)` will be helpful to locate the first occurence of a character in a string.
- Since HTTP is a text-based protocol, you might want to use the functions of the stream API. This has the advantage that you can process the message headers line by line by using for instance `fgets(3)` or `getline(3)`.
- The first line of both the request and the response headers of the HTTP consists of 3 values separated by whitespaces. You might find it helpful to separate these using `strtok(3)`.
- You might want to use `strftime(3)` to format date and time as required for the response header. Take a close look at the 'Example' section of the man page for advice on date formatting in compliance with various standards.
- Be aware that the server must the read the entire request from the connection socket before writing anything to that socket. This is also true if your server is responding with an error status.

Testing

In order to test the functionality and correct implementation of the protocol, you should first test your programs in following ways and then execute the testcases which are specified in the subsequent sections.

Assignment A – Your client requests files from servers on the internet

Using a web browser (e.g. Firefox), download a file which is publicly available on the internet, such as for instance `http://www.example.com/` and save it locally (e.g. under `~/Downloads/test.html`). Instead of a browser you can also use the command-line utility `wget`:

```
$ wget http://www.example.com/
```

Request the same file using your client and verify that the content of both files is identical using the utility `diff`.

```
$ ./client http://www.example.com/ > index.html
$ diff ~/Downloads/test.html index.html
$ mkdir dir
$ ./client -d dir http://www.example.com/index.html
$ diff ~/Downloads/test.html dir/index.html
```

Most websites recently refuse to transmit files via HTTP and instead require HTTPS, the encrypted version of HTTP. However, websites which still accept HTTP are for instance `http://neverssl.com/` or `http://www.example.com/`.

Assignment B – Your server processes requests from a web browser

Start your server and request files from it using a web browser (for instance using Firefox, which is the default browser in the lab). Again make sure that this works for various filetypes.

```
$ ls ./my_website
index.html
$ ./server ./my_website
```

When you type `127.0.0.1:8080` in the address bar of the browser (`127.0.0.1` is the IP address for localhost and `:8080` is used to specify the port), then your browser should request and display the file `index.html`. A browser will also request any requisites of a HTML document, such as images or scripts.

If you are working remotely via SSH you can also use `wget` instead of a browser with GUI:

```
$ wget -p -nd http://127.0.0.1:8080/
```

Assignment A and B – Your client requests files from your server

If you have implemented both, the server and the client, you may also test both programs together. Create a directory and populate it with one or more example files. Verify that requested files are transmitted correctly using the utility `diff`.

```
$ ls ./my_website
test.html
$ ./server -i test.html ./my_website &
$ ./client -o received.html -p 8080 http://localhost/
$ diff ./my_website/test.html received.html
```

Note that your implementation is not required to be able to transmit binary files (such as images).

Testcases

Input is shown in blue color. Output to *stdout* (and *stderr*) is shown in black. (Note that in the following output sections `EXIT_SUCCESS` equals `0`, and `EXIT_FAILURE` equals `1`. Refer to `stdlib.h` for further details.) `^C` indicates `CTRL+C`, `^D` indicates `CTRL+D`. The placeholder `<usage message>` must be replaced by a proper usage message (printed to *stdout*), `<error message>` must be replaced by a meaningful error message (which is printed to *stderr*). When testing at the lab infrastructure, you might need to change port `9999` to another value, if this port is already in use by another student.

A – HTTP Client

Some of these testcases use the built-in web server of PHP [4], which is very easy to use and available on the lab infrastructure.

A-Testcase 01: usage

```
$ ./client
<usage message>
$ echo $?
1
```

A-Testcase 02: invalid-port-1

```
$ ./client -p abc > /dev/null
<error message>
$ echo $?
1
```

A-Testcase 03: invalid-port-2

```
$ ./client -p 80x http://localhost/ > /dev/null
<error message>
$ echo $?
1
```

A-Testcase 04: invalid-options-1

```
$ ./client -p 80 -p 81 http://localhost/ > /dev/null
<error message>
$ echo $?
1
```

A-Testcase 05: invalid-options-2

```
$ ./client -a http://localhost/ > /dev/null
<error message>
$ echo $?
1
```

[4] PHP built-in web server: `https://www.php.net/manual/en/features.commandline.webserver.php`

A-Testcase 06: simple-1

Start HTTP Server first.

```
$ mkdir -p docroot
$ echo "Hello world." > docroot/hello.html
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [200]: /hello.html


^C
```

Then run your HTTP Client.

```


$ ./client -p 9999 http://localhost/hello.html
Hello world.
$ echo $?
0
```

A-Testcase 07: save-to-file-1

Start HTTP Server first.

```


$ mkdir -p docroot
$ echo "Index." > docroot/index.html
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [200]: /


^C
```

Then run your HTTP Client.

```
$ mkdir -p test1
$ rm -rf test1/*




$ ./client -p 9999 -d test1 http://localhost/
$ echo $?
0
$ cat test1/index.html
Index.
```

A-Testcase 08: save-to-file-2

Start HTTP Server first.

```

$ mkdir -p docroot
$ echo "This is the content." > docroot/t
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [200]: /t


^C
```

Then run your HTTP Client.

```
$ rm -rf f.2




$ ./client -p 9999 -o f.2 http://localhost/t
$ echo $?
0
$ cat f.2
This is the content.
```

A-Testcase 09: cannot-write-1

Start HTTP Server first.

```
1
2
3  $ mkdir -p docroot
4  $ echo "This is the content." > docroot/t
5  $ php -S 0.0.0.0:9999 -t docroot
6  PHP 5.4.16 Development Server started at ...
7  Listening on http://0.0.0.0:9999
8  Document root is /home/user/docroot
9  Press Ctrl-C to quit.
10 [...] ::1:45974 [200]: /t
11
12
13 ^C
```

Then run your HTTP Client.

```
1  $ touch f.3
2  $ chmod 000 f.3
3
4
5
6
7
8
9
10 $ ./client -p 9999 -o f.3 http://localhost/t > \
       /dev/null
11 <error message>
12 $ echo $?
13 1
```

A-Testcase 10: cannot-write-2

Start HTTP Server first.

```
1
2  $ mkdir -p docroot
3  $ echo "This is the content." > docroot/t
4  $ php -S 0.0.0.0:9999 -t docroot
5  PHP 5.4.16 Development Server started at ...
6  Listening on http://0.0.0.0:9999
7  Document root is /home/user/docroot
8  Press Ctrl-C to quit.
9  [...] ::1:45974 [200]: /t
10
11
12 ^C
```

Then run your HTTP Client.

```
1  $ rm -rf test.dir
2
3
4
5
6
7
8
9  $ ./client -p 9999 -d test.dir \
       http://localhost/t > /dev/null
10 <error message>
11 $ echo $?
12 1
```

A-Testcase 11: save-to-file-3

Start HTTP Server first.

```
1
2
3  $ mkdir -p docroot
4  $ echo "<?php echo 2*\$_GET[\"a\"] . \"\n\"; ?>" \
       > docroot/m.php
5  $ cat docroot/m.php
6  <?php echo 2*$_GET["a"] . "\n"; ?>
7  $ php -S 0.0.0.0:9999 -t docroot
8  PHP 5.4.16 Development Server started at ...
9  Listening on http://0.0.0.0:9999
10 Document root is /home/user/docroot
11 Press Ctrl-C to quit.
12 [...] ::1:45974 [200]: /m.php?a=3
13
14
15 ^C
```

Then run your HTTP Client.

```
1  $ mkdir -p test2
2  $ rm -rf test2/*
3
4
5
6
7
8
9
10
11
12 $ ./client -p 9999 -d test2 \
       http://localhost/m.php?a=3
13 $ echo $?
14 0
15 $ cat test2/m.php
16 6
```

Note: It would be preferred if you save the file as `m.php`, but since this bevaiour was not specified initially it is also ok, if you save it as `m.php?a=3`. In the latter case line 15 may be changed to `$>cat test2/m.php*`

A-Testcase 12: save-to-file-4

Start HTTP Server first.

```
1
2
3  $ mkdir -p docroot; rm docroot/*
4  $ echo "<?php print_r(\$_GET); ?>" > \
       docroot/index.php
5  $ cat docroot/index.php
6  <?php print_r($_GET); ?>
7  $ php -S 0.0.0.0:9999 -t docroot
8  PHP 5.4.16 Development Server started at ...
9  Listening on http://0.0.0.0:9999
10 Document root is /home/user/docroot
11 Press Ctrl-C to quit.
12 [...] ::1:45974 [200]: /?welcome=here
13
14
15 ^C
```

Then run your HTTP Client.

```
1  $ mkdir -p test3
2  $ rm -rf test3/*
3
4
5
6
7
8
9  $ ./client -p 9999 -d test3 \
       http://localhost?welcome=here
10 $ echo $?
11 0
12 $ cat test3/index.html
13 Array
14 (
15     [welcome] => here
16 )
```

Note: As the request in this case must contain `/?welcome=here`, but there is no slash after the host in the executed command (line 9), you may either add `/` in your client if needed, or change line 9 to `$>./client -p 9999 -d test3 http://localhost/?welcome=here`
Same note as for A-Testcase 11: save-to-file-3 also applies here.

A-Testcase 13: invalid-protocol

```
1  $ ./client -p 9999 foo://localhost > /dev/null
2  <error message>
3  $ echo $?
4  1
```

A-Testcase 14: invalid-hostname-1

```
1  $ ./client -p 9999 http:// > /dev/null
2  <error message>
3  $ echo $?
4  1
```

A-Testcase 15: invalid-hostname-2

```
1  $ ./client -p 9999 http://?getparam > /dev/null
2  <error message>
3  $ echo $?
4  1
```

A-Testcase 16: protocol-1

Start netcat first.

```
1  $ nc -l -p 9999 -c "echo -e \"HTTP/1.1 200 \
       OK\r\n\r\nNetcat.\""
```

Then run your HTTP Client.

```
1
2  $ ./client -p 9999 http://localhost/
3  Netcat.
4  $ echo $?
5  0
```

A-Testcase 17: protocol-2

Start netcat first.

```
$ nc -l 9999 --crlf
GET / HTTP/1.1
Host: localhost
Connection: close

test

^D
```

Then run your HTTP Client.

```

$ ./client -p 9999 http://localhost/ > /dev/null
Protocol error!
$ echo $?
2
```

A-Testcase 18: protocol-3

Start netcat first.

```
$ nc -l 9999 --crlf
GET / HTTP/1.1
Host: localhost
Connection: close

HTTP/1.1

^D
```

Then run your HTTP Client.

```

$ ./client -p 9999 http://localhost/ > /dev/null
Protocol error!
$ echo $?
2
```

A-Testcase 19: protocol-4

Start netcat first.

```
$ nc -l 9999 --crlf
GET / HTTP/1.1
Host: localhost
Connection: close

HTTP/1.1 foobar

^D
```

Then run your HTTP Client.

```

$ ./client -p 9999 http://localhost/ > /dev/null
Protocol error!
$ echo $?
2
```

A-Testcase 20: protocol-5

Start netcat first.

```
$ nc -l 9999 --crlf
GET / HTTP/1.1
Host: localhost
Connection: close

HTTP/1.1 200 OK

Hello.
^D
```

Then run your HTTP Client.

```

$ ./client -p 9999 http://localhost/
Hello.
$ echo $?
0
```

A-Testcase 21: no-server

Do not start HTTP Server.

Only run your HTTP Client.

```
$ ./client -p 9999 http://localhost/ > /dev/null
<error message>
$ echo $?
1
```

A-Testcase 22: status-code-1

Start HTTP Server first.

```
$ mkdir -p docroot; rm docroot/*
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [404]: /hello.html - No such \
    file or directory
^C
```

Then run your HTTP Client.

```
$ ./client -p 9999 http://localhost/hello.html > \
    /dev/null
404 Not Found
$ echo $?
3
```

A-Testcase 23: status-code-2

Start netcat first.

```
$ nc -l -p 9999 -c "echo -e \"HTTP/1.1 418 Tea \
    cold\r\n\r\nNetcat.\""
```

Then run your HTTP Client.

```
$ ./client -p 9999 http://localhost/ > /dev/null
418 Tea cold
$ echo $?
3
```

A-Testcase 24: simple-2

Start HTTP Server first.

```
$ mkdir -p docroot
$ echo "Hello user." > docroot/hello.html
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [200]: /hello.html


^C
```

Then run your HTTP Client.

```
$ ./client -p 9999 http://127.0.0.1/hello.html
Hello user.
$ echo $?
0
```

A-Testcase 25: lines-1

Start HTTP Server first.

```
$ mkdir -p docroot
$ ( for i in `seq 1 10`; do echo "ABC$i" | \
    sha1sum | cut -d " " -f1 | tr -d \\n; printf \
    -- "-%.0s" {1..8000}; echo "__end$i."; done; \
    ) > docroot/longlines.html
$ sha1sum docroot/longlines.html
c1d5737e1fdb7fab94c30a3ab0fe44aa775c2bb3  \
    longlines.html
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [200]: /longlines.html


^C
```

Then run your HTTP Client.

```
$ ./client -p 9999 -o lines1.html \
    http://127.0.0.1/longlines.html
$ echo $?
0
$ sha1sum lines1.html
c1d5737e1fdb7fab94c30a3ab0fe44aa775c2bb3  \
    lines1.html
```

A-Testcase 26: lines-2

Start HTTP Server first.

```
$ mkdir -p docroot
$ echo -n "01234567890abcdefghijklmnopqr" > chars
$ echo -n "stuvwxyz01234567890abcdefghij" >> chars
$ echo -n "klmnopqrstuvwxyz.,:-!=?% ABCD" >> chars
$ echo -n "EFGHIJKLMNOPQRSTUVWXYZABCDEFG" >> chars
$ echo -n "HIJKLMNOPQRSTUVWXYZ" >> chars
$ ( for i in `cat chars | sed "s/./\0\n/g"`; do \
    echo $i | sha1sum | tr -d "\n"; echo -n "x"; \
    cat chars | sed "s/./\0$i/g"; echo ""; done; \
    ) > docroot/mixed.html
$ sha1sum docroot/mixed.html
966debac96d8b1b40715666c93328bd08e6ecc5d  \
    mixed.html
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [200]: /mixed.html


^C
```

Then run your HTTP Client.

```
$ ./client -p 9999 -o lines2.html \
    http://127.0.0.1/mixed.html
$ echo $?
0
$ sha1sum lines2.html
966debac96d8b1b40715666c93328bd08e6ecc5d  \
    lines2.html
```

A-Testcase 27: protocol-5

Start netcat first.

```
$ nc -l -p 9999 -c "echo \
    \"0iZRefnSwfL62th125FKNVvlKuY=\" | base64 -d"
```

Then run your HTTP Client.

```

$ ./client -p 9999 http://localhost/
Protocol error!
$ echo $?
2
```

A-Testcase 28: invalid-port-3

```
$ ./client -p -2 http://localhost/ > /dev/null
<error message>
$ echo $?
1
```

A-Testcase 29: invalid-port-4

```
$ ./client -p 65599 http://localhost/ > /dev/null
<error message>
$ echo $?
1
```

B – HTTP Server

B-Testcase 01: usage

```
$ ./server
<usage message>
$ echo $?
1
```

B-Testcase 02: invalid-port-1

```
$ ./server -p abc . > /dev/null
<error message>
$ echo $?
1
```

B-Testcase 03: invalid-port-2

```
$ ./server -p 80z . > /dev/null
<error message>
$ echo $?
1
```

B-Testcase 04: invalid-options-1

```
$ ./server -p 80 -p 81 . > /dev/null
<error message>
$ echo $?
1
```

B-Testcase 05: invalid-options-2

```
$ ./server -a . > /dev/null
<error message>
$ echo $?
1
```

B-Testcase 06: simple-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "Hello world." > docroot/hello.html
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then run the HTTP Client.

```



$ wget http://localhost:9999/hello.html -O - \
    2>/dev/null
Hello world.
$ echo $?
0
```

B-Testcase 07: simple-2

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "Index." > docroot/index.html
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then run the HTTP Client.

```


$ wget http://localhost:9999 -O - 2>/dev/null
Index.
$ echo $?
0
```

B-Testcase 08: simple-3

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "Main." > docroot/main.file
$ ./server -p 9999 -i main.file docroot




^C
$ echo $?
0
```

Then run the HTTP Client.

```


$ wget http://localhost:9999 -O - 2>/dev/null
Main.
$ echo $?
0
```

B-Testcase 09: protocol-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "12345" > docroot/123
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ echo -ne "GET /123 HTTP/1.1\r\n\r\n" | nc \
    localhost 9999
HTTP/1.1 200 OK
Date: Thu, 1 Dec 20 12:00:00
Content-Length: 6
Connection: close

12345
```

B-Testcase 10: not-found-1

Start your HTTP Server first.

```
$ mkdir -p docroot; rm docroot/*
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ echo -ne "GET /file HTTP/1.1\r\n\r\n" | nc \
    localhost 9999
HTTP/1.1 404 (Not Found)
Connection: close
```

B-Testcase 11: not-found-2

Start your HTTP Server first.

```
$ mkdir -p docroot; rm docroot/*
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ wget http://localhost:9999/test
...
HTTP request sent, awaiting response... 404 (Not \
    Found)
...
$ echo $?
8
```

B-Testcase 12: invalid-protocol-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ echo -ne "TRACE /file HTTP/1.1\r\n\r\n" | nc \
    localhost 9999
HTTP/1.1 501 (Not Implemented)
Connection: close
```

B-Testcase 13: invalid-protocol-2

Start your HTTP Server first.

```
$ mkdir -p docroot
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ wget http://localhost:9999/test --post-data=
...
HTTP request sent, awaiting response... 501 (Not \
    Implemented)
...
$ echo $?
8
```

Note: This testcase was updated to match the submission server.
(Before it indicated: `wget http://localhost:9999/test --post-data=test`).

B-Testcase 14: invalid-protocol-3

Start your HTTP Server first.

```
$ mkdir -p docroot
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ echo -ne "test\r\n\r\n" | nc localhost 9999
HTTP/1.1 400 (Bad Request)
Connection: close
```

B-Testcase 15: invalid-protocol-4

Start your HTTP Server first.

```
$ mkdir -p docroot
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ echo -ne "GET path HTTP/9.9\r\n\r\n" | nc \
    localhost 9999
HTTP/1.1 400 (Bad Request)
Connection: close
```

B-Testcase 16: invalid-protocol-5

Start your HTTP Server first.

```
$ mkdir -p docroot
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ echo -ne "a b c d e f\r\n\r\n" | nc localhost \
    9999
HTTP/1.1 400 (Bad Request)
Connection: close
```

B-Testcase 17: invalid-protocol-6

Start your HTTP Server first.

```
$ mkdir -p docroot
$ ./server -p 9999 docroot




^C
$ echo $?
0
```

Then send the request.

```


$ (echo "BkMqIN91b9DZhr5b50bfmYkfUwA=" | base64 \
    -d; echo -ne "\r\n\r\n") | nc localhost 9999
HTTP/1.1 400 (Bad Request)
Connection: close
```

Note: This testcase was updated to match the submission server (added: `echo -ne "\r\n\r\n"`).

B-Testcase 18: multi-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "Hello world." > docroot/hello.html
$ ./server -p 9999 docroot









^C
$ echo $?
0
```

Then run the HTTP Client.

```



$ wget http://localhost:9999/hello.html -O - \
    2>/dev/null
Hello world.
$ wget http://localhost:9999/hello.html -O - \
    2>/dev/null
Hello world.
$ wget http://localhost:9999/hello.html -O - \
    2>/dev/null
Hello world.
```

B-Testcase 19: signal-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "12345" > docroot/123
$ ./server -p 9999 docroot

^C









$ echo $?
0
```

Then send the request.

```


$ nc localhost 9999 --crlf
GET /123 HTTP/1.1

HTTP/1.1 200 OK
Date: Thu, 1 Dec 20 12:00:00
Content-Length: 6
Connection: close

12345
```

Order of commands/interaction: First start the server (line 3 left). Then start netcat (line 4 right) without sending any data yet. Then send `SIGINT` (using `^C`, line 5 left). Note: The server does not terminate, as there is still a request ongoing. Then send data (line 5 right).

B-Testcase 20: lines-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ ( for i in `seq 1 10`; do echo "ABC$i" | \
    sha1sum | cut -d " " -f1 | tr -d \\n; printf \
    -- "-%.0s" {1..8000}; echo "__end$i."; done; \
    ) > docroot/longlines.html
$ sha1sum docroot/longlines.html
c1d5737e1fdb7fab94c30a3ab0fe44aa775c2bb3  \
    longlines.html
$ ./server -p 9999 docroot



^C
```

Then run HTTP Client.

```




$ wget http://localhost:9999/longlines.html -O \
    lines1 2>/dev/null
$ echo $?
0
$ sha1sum lines1
c1d5737e1fdb7fab94c30a3ab0fe44aa775c2bb3  lines1
```

B-Testcase 21: lines-2

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo -n "01234567890abcdefghijklmnopqr" > chars
$ echo -n "stuvwxyz01234567890abcdefghij" >> chars
$ echo -n "klmnopqrstuvwxyz.,:-!=?% ABCD" >> chars
$ echo -n "EFGHIJKLMNOPQRSTUVWXYZABCDEFG" >> chars
$ echo -n "HIJKLMNOPQRSTUVWXYZ" >> chars
$ ( for i in `cat chars | sed "s/./\0\n/g"`; do \
    echo $i | sha1sum | tr -d "\n"; echo -n "x"; \
    cat chars | sed "s/./\0$i/g"; echo ""; done; \
    ) > docroot/mixed.html
$ sha1sum docroot/mixed.html
966debac96d8b1b40715666c93328bd08e6ecc5d  \
    mixed.html
$ ./server -p 9999 docroot

^C
```

Then run HTTP Client.

```




$ wget http://localhost:9999/mixed.html -O \
    lines2 2>/dev/null
$ sha1sum lines2
966debac96d8b1b40715666c93328bd08e6ecc5d  lines2
```

B-Testcase 22: invalid-port-3

```
$ ./server -p -2 docroot > /dev/null
<error message>
$ echo $?
1
```

B-Testcase 23: invalid-port-4

```
$ ./server -p 65599 docroot > /dev/null
<error message>
$ echo $?
1
```

Optional testcases

A – HTTP Client

A-Testcase 51: binary-1

Start HTTP Server first.

```
$ mkdir -p docroot
$ rm docroot/bin; for i in {0..255}; do printf \
    "0x%02X\n" $i | xxd -r >> docroot/bin; done;
$ sha1sum docroot/bin
4916d6bdb7f78e6803698cab32d1586ea457dfc8  \
    docroot/bin
$ php -S 0.0.0.0:9999 -t docroot
PHP 5.4.16 Development Server started at ...
Listening on http://0.0.0.0:9999
Document root is /home/user/docroot
Press Ctrl-C to quit.
[...] ::1:45974 [200]: /bin


^C
```

Then run your HTTP Client.

```


$ ./client -p 9999 -o binout http://localhost/bin
$ echo $?
0
$ sha1sum binout
4916d6bdb7f78e6803698cab32d1586ea457dfc8  binout
```

B – HTTP Server

B-Testcase 51: binary-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ rm docroot/bin; for i in {0..255}; do printf \
    "0x%02X\n" $i | xxd -r >> docroot/bin; done;
$ sha1sum docroot/bin
4916d6bdb7f78e6803698cab32d1586ea457dfc8  \
    docroot/bin
$ ./server -p 9999 docroot



^C
$ echo $?
0
```

Then run the HTTP Client.

```


$ wget http://localhost:9999/bin -O bincl
$ sha1sum bincl
4916d6bdb7f78e6803698cab32d1586ea457dfc8  bincl
```

B-Testcase 52: content-type-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "content of index.html." > \
    docroot/index.html
$ echo "content of test.htm." > docroot/test.htm
$ echo "content of style.css." > docroot/style.css
$ echo "content of lib.js." > docroot/lib.js
$ echo "content of other.file." > \
    docroot/other.file
$ ./server -p 9999 docroot
```

```
^C
$ echo $?
0
```

Then run the HTTP Client.

```
$ wget http://localhost:9999/ -q -S -O -
  HTTP/1.1 200 OK
  Date: Thu, 1 Dec 20 12:00:00
  Content-Length: 23
  Connection: close
  Content-Type: text/html
content of index.html.
$ wget http://localhost:9999/index.html -q -S -O -
  HTTP/1.1 200 OK
  Date: Thu, 1 Dec 20 12:00:00
  Content-Length: 23
  Connection: close
  Content-Type: text/html
content of index.html.
$ wget http://localhost:9999/test.htm -q -S -O -
  HTTP/1.1 200 OK
  Date: Thu, 1 Dec 20 12:00:00
  Content-Length: 21
  Connection: close
  Content-Type: text/html
content of test.htm.
$ wget http://localhost:9999/style.css -q -S -O -
  HTTP/1.1 200 OK
  Date: Thu, 1 Dec 20 12:00:00
  Content-Length: 22
  Connection: close
  Content-Type: text/css
content of style.css.
$ wget http://localhost:9999/lib.js -q -S -O -
  HTTP/1.1 200 OK
  Date: Thu, 1 Dec 20 12:00:00
  Content-Length: 19
  Connection: close
  Content-Type: application/javascript
content of lib.js.
$ wget http://localhost:9999/other.file -q -S -O -
  HTTP/1.1 200 OK
  Date: Thu, 1 Dec 20 12:00:00
  Content-Length: 23
  Connection: close
content of other.file.
```

Additional testcases

These testcases are not executed by the automatic submission evaluation system, but might be helpful for manual testing.

B – HTTP Server

B-Testcase: response-detail-1

Start your HTTP Server first.

```
$ mkdir -p docroot
$ echo "Index." > docroot/index.html
$ ./server -p 9999 docroot









^C
$ echo $?
0
```

Then send the request.

```


$ echo -e "GET / HTTP/1.1\r\n\r\n" | nc \
     localhost 9999 | grep "HTTP/1.1" | xxd -c 10
0000000: 4854 5450 2f31 2e31 2032  HTTP/1.1 2
000000a: 3030 204f 4b0d 0a         00 OK..
```

Note: This testcase was updated to include the proper end for the request header (`\r\n\r\n`).

Recommendations and Guidelines

In order to make all solutions more uniform and to avoid problems with different C dialects, please try to follow the following guidelines.

General Guidelines

Try to follow the following guidelines closely.

1. All source files of your program(s) should compile with
   ```
   $ gcc -std=c99 -pedantic -Wall -D_DEFAULT_SOURCE -D_BSD_SOURCE -D_SVID_SOURCE
                                  -D_POSIX_C_SOURCE=200809L -g -c filename.c
   ```
   without *warnings and info messages* and your program(s) should link without warnings too.
2. There should be a Makefile implementing the targets: `all` to build the program(s) (i.e. generate executables) from the sources (this must be the first target in the Makefile); `clean` to delete all files that can be built from your sources with the Makefile.
3. Make all targets of your Makefile idempotent, i.e. execution of `make clean; make clean` should yield the same result as `make clean`, and should not fail with an error.
4. The program shall operate according to the specification/assignment without major issues (e.g., segmentation fault, memory corruption).
5. Arguments should be parsed according to UNIX conventions (we strongly encourage the use of `getopt(3)`). The program should conform to the given synopsis/usage in the assignment. If the synopsis is violated (e.g., unspecified options or too many arguments), the program should terminate with the usage message containing the program name and the correct calling syntax. Argument handling should also be implemented for programs without arguments.
6. Correct (=normal) termination should include a cleanup of resources.
7. Upon success the program shall terminate with exit code 0, in case of errors with an exit code greater than 0. We recommend to use the macros `EXIT_SUCCESS` and `EXIT_FAILURE` (defined in `stdlib.h`) to enable portability of the program.
8. If a function indicates an error with its return value, it *should* be checked in general. If the subsequent code depends on the successful execution of a function (e.g., resource allocation), it is very *important* to check the return value.
9. Functions that do not take any parameters should be declared with `void` in the signature, e.g., `int get_random_int(void);`.
10. Procedures (i.e., functions that do not return a value) should be declared as `void`.
11. Error messages shall be written to `stderr` and should contain the program name `argv[0]`.
12. Do not use the following functions to avoid crashes due to invalid inputs:
    - `gets(3)`; use `fgets(3)` instead.
    - `scanf(3)` and `fscanf(3)`; use `fgets(3)` and `sscanf(3)` instead.
    - `atoi(3)` and `atol(3)`; use `strtol(3)` instead.
13. Documentation is a nice bonus. Format the documentation in Doxygen style (see Wiki and Doxygen's intro).

14. Write meaningful comments. For example, meaningful comments describe the algorithm, or why a particular solution has been chosen, if there seems to be an easier solution at a first glance. Avoid comments that just repeat the code itself
(e.g., `i = i + 1; /* i is incremented by one */`).

15. The documentation of a module should include: name of the module, name and student id of the author (`@author` tag), purpose of the module (`@brief`, `@details` tags) and creation date of the module (`@date` tag).

    The Makefile should also include a header, with author and program name at least.

16. Each function shall be documented either before the declaration or the implementation. It should include purpose (`@brief`, `@details` tags), description of parameters and return value (`@param`, `@return` tags) and description of global variables the function uses (`@details` tag).

    You should also document `static` functions (see `EXTRACT_STATIC` in the file `Doxyfile`). Document visible/exported functions in the header file and local (`static`) functions in the C file. Document variables, constants and types (especially `structs`) too.

17. Documentation, names of variables and constants is best done in English.

18. Internal functions shall be marked with the `static` qualifier and are not allowed to be exported (e.g., in a header file). Only functions that are used by other modules shall be declared in the header file.

19. All exercises shall be solved with functions of the C standard library. If a required function is not available in the standard library, you can use other (external) functions too. Avoid reinventing the wheel (e.g., re-implementation of `strcmp`).

20. Name of constants shall be written in upper case, names of variables in lower case (possibly with the first letter capitalized).

21. Use meaningful variable and constant names (e.g., also semaphores and shared memories).

22. Avoid using global variables as far as possible.

23. All boundaries shall be defined as constants (macros). Avoid arbitrary boundaries. If boundaries are necessary, treat its crossing.

24. Avoid side effects with `&&` and `||`, e.g., write `if(b != 0) c = a/b;` instead of `if(b != 0 && c = a/b)`.

25. Each `switch` block should contain a `default` case. If the case is not reachable, write `assert(0)` to this case (defensive programming).

26. Logical values shall be treated with logical operators, numerical values with arithmetic operators (e.g., test 2 strings for equality by `strcmp(...) == 0` instead of `!strcmp(...)`).

27. Indent your source code consistently (there are tools for that purpose, e.g., `indent`).

28. Avoid tricky arithmetic statements. Programs are written once, but read many times. Your program is not always better if it is shorter!

29. For all I/O operations (read/write from/to `stdin`, `stdout`, files, sockets, pipes, etc.) use *either* standard I/O functions (`fdopen(3)`, `fopen(3)`, `fgets(3)`, etc.) *or* POSIX functions (`open(2)`, `read(2)`, `write(2)`, etc.). Remember, standard I/O functions are buffered. Mixing standard I/O functions and POSIX functions to access a common file descriptor can lead to undefined behaviour and is therefore a bad idea.

30. If asked in the assignment, you should implement signal handling (`SIGINT`, `SIGTERM`). You must use only *async-signal-safe* functions in your signal handlers.

31. Close files, free dynamically allocated memory, and remove resources after usage.

32. Don't waste resources due to inconvenient programming. Header files shall not include implementation parts (exception: macros).

33. To comply with the given testcases, the program output must exactly match the given specification. Therefore you should only print debug information if the compile flag `-DDEBUG` is set, in order to avoid problems with the automatic checker.

Exercise 3 Guidelines

The following points are related for Exercise 3.

1. It is discouraged to use the deprecated functions `gethostbyname`, `gethostbyaddr`, `gethostbyname2`, `gethostbyname_r`, `gethostbyname2_r` or any other function of that family documented on the man page `gethostbyname(3)`. Use `getaddrinfo(3)` instead.